Module 18: Java Threads, Executor Framework, CompletableFuture, Spring Boot Scheduling, Async, and Tomcat Threading Model

Questions

Q1. What is a thread in Java, and how does it differ from a process?

Answer: A thread is the smallest unit of execution within a process. While a process has its own isolated memory space, file descriptors, and system resources, threads within the same process share the heap memory and most resources but maintain separate program counters, registers, and stack memory. Multiple threads allow concurrent execution paths within a single application. In Java, threads are managed by the JVM and mapped to operating system threads through the underlying platform. The main advantage of threads over separate processes is lower creation and context-switching overhead because they share memory. The major risk is that shared memory requires careful synchronization to avoid race conditions, stale reads, and inconsistent state. Real-time example: consider an e-commerce web server handling many customer requests. If each request ran in a separate process, the server would consume enormous memory and CPU just for process management. Instead, the server uses a thread pool where each request runs in a thread. All threads share the connection pool and cache, but each has its own stack for local variables. When two threads try to update the same inventory counter simultaneously, synchronization is required to prevent overselling.

Q2. What are the ways to create a thread in Java?

Answer: There are two primary ways to create a thread. The first is to extend the `Thread` class and override the `run()` method, then call `start()`. The second, preferred approach is to implement the `Runnable` interface and pass an instance to a `Thread` constructor. The `Runnable` approach is preferred because it separates the task logic from the thread mechanism and avoids the single-inheritance limitation of extending `Thread`. Since Java 5, a third way is to implement `Callable` and submit it to an `ExecutorService`, which returns a `Future` for result retrieval. Java 8 introduced lambda expressions, making `Runnable` and `Callable` creation concise. A backend developer should prefer `Runnable`, `Callable`, and executors over raw `Thread` creation because executors manage thread lifecycle and reuse. Real-time example: a beginner might write `new Thread(new MyTask()).start()` for a background job. In a production system, this is discouraged because creating and destroying threads constantly is expensive. Instead, a senior developer writes:

```
executorService.submit(() -> processOrder(orderId));
```

This submits a `Runnable` to a pool that reuses threads, handles rejection, and can be shut down gracefully.

Q3. What is the difference between `run()` and `start()`?

Answer: Calling `start()` on a `Thread` object begins a new thread of execution and invokes the `run()` method asynchronously in that newly created thread. Calling `run()` directly does not start a new thread; it executes the method synchronously in the current thread just like any other method call, which defeats the purpose of multithreading. Starting a thread twice by calling `start()` twice throws `IllegalThreadStateException`. The `run()` method is the entry point containing the task logic, while `start()` is the mechanism that schedules the thread with the JVM and underlying operating system. Real-time example: a junior developer writes:

```
Thread t = new Thread(myRunnable);
t.run();
```

Expecting the task to run in the background, but the code actually blocks the calling thread. The fix is `t.start()`. Another common mistake is calling `start()` on the same `Thread` instance twice, which throws `IllegalThreadStateException`. The correct pattern is to create a new `Thread` instance for each new execution.

Q4. What are the states in the Java thread lifecycle?

Answer: Java threads transition through several states that are accessible through `Thread.State`. `NEW` means the thread is created but `start()` has not been called. `RUNNABLE` means the thread is ready to run or currently running on a CPU. `BLOCKED` means the thread is waiting to acquire a monitor lock. `WAITING` means the thread is waiting indefinitely for another thread to perform an action, such as through `Object.wait()` without a timeout or `Thread.join()` without a timeout. `TIMED_WAITING` means the thread is waiting for a specified time, such as through `Thread.sleep(millis)`, `Object.wait(timeout)`, or `Thread.join(timeout)`. `TERMINATED` means execution has completed. Understanding the lifecycle is essential for diagnosing deadlocks, thread starvation, and performance issues in production by analyzing thread dumps. Real-time example: a production service hangs during peak load. A thread dump reveals many threads in `BLOCKED` state waiting for a lock held by a single thread in `RUNNABLE` state that is doing heavy computation. This points to a lock contention bottleneck. Reducing the scope of the synchronized block reduces the `BLOCKED` time and restores throughput.

Q5. What is the difference between `sleep()`, `yield()`, and `wait()`?

Answer: `Thread.sleep(millis)` pauses the current thread for a specified time without releasing any locks it currently holds. It transitions the thread to `TIMED_WAITING`. `Thread.yield()` is a hint to the scheduler that the current thread is willing to give up the CPU, but it is not guaranteed to pause and does not release locks. `Object.wait()` causes the current thread to release the object's monitor lock and wait until another thread calls `notify()` or `notifyAll()` on the same object, transitioning the thread to `WAITING` or `TIMED_WAITING`. `wait()` must be called inside a synchronized block or method because it requires ownership of the

object's monitor to release it. In contrast, `sleep()` and `yield()` are static methods on `Thread` and do not require synchronization. Real-time example: in a producer-consumer queue implemented manually, the consumer calls `wait()` when the queue is empty, releasing the lock so the producer can add items. The producer calls `notifyAll()` after adding an item. If the consumer used `sleep()` instead of `wait()`, it would hold the lock while sleeping, blocking the producer from adding items, which would deadlock the system.

Q6. What is the Java Memory Model, and why is it important for concurrency?

Answer: The Java Memory Model defines the rules for how threads interact through memory and what behaviors the Java language guarantees. It specifies visibility, ordering, and atomicity rules for shared variable access. Without the JMM, different threads might see different values of the same variable due to CPU caches, compiler optimizations, and instruction reordering. The JMM provides happens-before relationships through synchronization, `volatile` writes and reads, `Thread.start()`, `Thread.join()`, and methods in the `java.util.concurrent` classes. Understanding the JMM is essential for writing correct multithreaded programs and avoiding subtle bugs such as stale reads, partially constructed objects, and reordered instructions. Real-time example: one thread sets a boolean flag `boolean running = true` to signal a worker thread to stop. Without `volatile` or synchronization, the worker thread may cache the old value `false` indefinitely and never exit. Declaring the flag as `volatile` creates a happens-before relationship so the worker sees the update. The JMM is the formal foundation that makes this guarantee possible.

Q7. What does the `volatile` keyword do?

Answer: The `volatile` keyword ensures that reads and writes of a variable are always performed against main memory, providing visibility of changes across threads. It also prevents certain instruction reorderings around the volatile access, establishing happens-before relationships. However, `volatile` does not provide atomicity for compound operations such as increment, which involves read-modify-write. A `volatile` variable is useful for simple flags that one thread writes and other threads read, such as a shutdown signal or a configuration toggle. For read-modify-write operations, `java.util.concurrent.atomic` classes such as `AtomicInteger` or synchronization should be used instead. Real-time example: a background processing loop uses a `boolean running` flag to control whether it should continue. Declaring it as `volatile boolean running = true;` ensures that when the main thread sets `running = false`, the background thread sees the change immediately and exits cleanly. If `volatile` is omitted, the background thread might keep running forever because it reads a cached copy of the flag.

Q8. What is the difference between `synchronized` and `ReentrantLock`?

Answer: `synchronized` is a built-in language-level locking mechanism that automatically acquires and releases a monitor lock. It is simple to use and reentrant, meaning a thread can reacquire a lock it already holds, but it has limited features. `ReentrantLock` is a class in `java.util.concurrent.locks` that provides the same basic mutual exclusion but with additional capabilities such as `tryLock()` with timeout, `lockInterruptibly()` for interruptible lock acquisition, a fairness policy option, and multiple `Condition` objects per lock. `ReentrantLock` requires explicit `lock()` and `unlock()` calls, usually wrapped in a try-finally block to ensure release. Use `synchronized` for simple, straightforward locking because it is less error-prone. Use `ReentrantLock` when you need advanced features such as timed lock attempts, fairness, or multiple wait sets. Real-time example: a banking transfer service needs to acquire locks on two accounts without blocking forever if one is unavailable. `synchronized` cannot attempt acquisition with a timeout. `ReentrantLock` allows `accountA.lock.tryLock(100, TimeUnit.MILLISECONDS)` and a rollback strategy if the second lock cannot be acquired, preventing deadlock and improving liveness.

Q9. What is the difference between a `Runnable` and a `Callable`?

Answer: `Runnable` represents a task that does not return a result and cannot throw checked exceptions from its `run()` method. It is suitable for fire-and-forget background tasks. `Callable` represents a task that returns a result and can throw checked exceptions from its `call()` method. `Callable` tasks are submitted to an `ExecutorService`, which returns a `Future` that can be used to retrieve the result, check completion, or handle exceptions. Use `Runnable` for tasks where the caller does not need to know the outcome, such as logging or cleanup. Use `Callable` when the caller needs a computed value or must handle errors, such as fetching data from a remote service. Real-time example: a batch payment processor needs to process each payment and know whether it succeeded. Using `Callable<PaymentResult>` for each payment and collecting `Future<PaymentResult>` objects allows the system to determine which payments succeeded and which failed. With `Runnable`, failures would be lost unless extra mechanisms were built.

Q10. What is the Executor Framework?

Answer: The Executor Framework, introduced in Java 5, decouples task submission from task execution mechanics. It provides a standard way to create and manage thread pools through the `Executor`, `ExecutorService`, and `ScheduledExecutorService` interfaces. The framework handles thread creation, reuse, scheduling, lifecycle management, and shutdown. It includes factory methods in `Executors` for common pool types such as fixed, cached, single, scheduled, and work-stealing pools, as well as the `ThreadPoolExecutor` class for fully custom configuration. Using executors is strongly preferred over manual `Thread` creation because it improves performance through thread reuse, scalability through controlled concurrency, and maintainability through consistent management policies. Real-time example: an application that

processes customer uploads might naively create a new thread per upload. At 100 uploads per second, the JVM exhausts native memory. Replacing this with a `ThreadPoolExecutor` of 20 threads and a bounded queue of 500 tasks allows the system to process uploads at a sustainable rate, apply backpressure through `CallerRunsPolicy`, and shut down gracefully during deployment.

Q11. How does `ThreadPoolExecutor` work?

Answer: `ThreadPoolExecutor` maintains a pool of worker threads and a work queue. When a task is submitted, the pool creates a new worker thread if the number of running threads is below `corePoolSize`, even if existing threads are idle. If the number of running threads is at least `corePoolSize`, the task is placed in the work queue. If the queue is full and the thread count is below `maximumPoolSize`, the pool creates additional threads up to the maximum. If the thread count has reached `maximumPoolSize` and the queue is full, the task is rejected according to the configured `RejectedExecutionHandler`. Threads above `corePoolSize` that remain idle for longer than `keepAliveTime` are terminated unless `allowCoreThreadTimeOut(true)` is set. This design provides predictable resource usage while allowing temporary bursts above the core size. Real-time example: an order processing system configures `corePoolSize=20`, `maximumPoolSize=100`, a bounded queue of 500, and `keepAliveTime=60s`. During normal hours, 20 threads handle the load. During a flash sale, the queue fills and the pool expands to 100 threads. After the sale, idle extra threads are terminated, returning to 20 threads and saving resources.

Q12. What are the different work queues and their effects on `ThreadPoolExecutor`?

Answer: The choice of work queue dramatically changes pool behavior. `SynchronousQueue` has no capacity and directly hands off tasks to threads; tasks are rejected if no thread is immediately available, so `maximumPoolSize` must be large enough. `LinkedBlockingQueue` can be bounded or unbounded; an unbounded queue means tasks never overflow to extra threads above `corePoolSize`, so `maximumPoolSize` is effectively ignored, but tasks can accumulate indefinitely and cause out-of-memory errors. `ArrayBlockingQueue` is bounded and balanced, allowing the pool to grow to `maximumPoolSize` before rejecting.
`PriorityBlockingQueue` orders tasks by priority and is unbounded. The queue choice determines backpressure, memory usage, and throughput characteristics. Real-time example: `Executors.newFixedThreadPool(10)` uses an unbounded `LinkedBlockingQueue`. If the producer submits tasks faster than the pool processes them, the queue grows until the JVM runs out of memory. A production-grade pool uses `ArrayBlockingQueue` with a fixed capacity of 1000 and `CallerRunsPolicy` so the producer slows down when saturated, providing natural backpressure.

Q13. What are the standard rejection policies in `ThreadPoolExecutor`?

Answer: The standard rejection policies are `AbortPolicy`, `CallerRunsPolicy`, `DiscardPolicy`, and `DiscardOldestPolicy`. `AbortPolicy` is the default and throws `RejectedExecutionException`, making the failure visible to the caller. `CallerRunsPolicy` runs the rejected task in the caller's thread, providing natural backpressure by slowing down the submitter. `DiscardPolicy` silently drops the task, which can lead to data loss and should rarely be used. `DiscardOldestPolicy` discards the oldest queued task and retries submission, which can cause starvation of old tasks. Custom policies can be implemented for logging, metrics, or alternate handling such as persisting the task for later retry. Real-time example: an API gateway forwards requests to a backend worker pool. Under overload, `AbortPolicy` would cause cascading failures as callers receive exceptions. Switching to `CallerRunsPolicy` makes the gateway thread process the request itself, slowing down the gateway and giving the backend time to recover. This is a classic backpressure pattern.

Q14. What is `Future`, and what are its limitations?

Answer: `Future` represents the result of an asynchronous computation. It provides methods to check if the computation is complete with `isDone()`, wait for completion with `get()`, retrieve the result with `get()`, and cancel the task with `cancel()`. Its limitations include that `get()` blocks the calling thread, there is no easy way to compose multiple futures, there is no built-in exception handling chain, and there is no direct support for callbacks upon completion. These limitations make `Future` cumbersome for complex asynchronous workflows. Java 8 introduced `CompletableFuture` to address these gaps by providing a rich functional, non-blocking API for composition, transformation, error handling, and callbacks. Real-time example: a service calls three backend services and needs to combine results. With plain `Future`, the code blocks on each `get()` sequentially and the exception handling is verbose. With `CompletableFuture`, the code chains `supplyAsync` calls and combines them with `thenCombine`, improving readability and parallelism.

Q15. What is `CompletableFuture`, and how does it differ from `Future`?

Answer: `CompletableFuture` is an implementation of both `Future` and `CompletionStage` that supports non-blocking composition of asynchronous operations. It allows chaining with methods such as `thenApply`, `thenCompose`, `thenCombine`, `handle`, `exceptionally`, `whenComplete`, and `allOf`. It supports synchronous stages that run on the completion thread and asynchronous stages via `*Async` methods that run on the `ForkJoinPool.commonPool()` or a supplied custom executor. Unlike `Future`, `CompletableFuture` can be explicitly completed with a value or exception using `complete()` or `completeExceptionally()`, supports callbacks without blocking, and enables complex pipelines. It is the preferred tool for modern asynchronous Java programming. Real-time example: an order placement API needs to validate inventory, reserve payment, and create shipment in sequence. With `CompletableFuture`, the code reads as:

```
validateInventory(order)
    .thenCompose(valid -> reservePayment(order))
    .thenCompose(paid -> createShipment(order))
    .thenApply(shipment -> OrderResult.success(shipment))
    .exceptionally(ex -> OrderResult.failure(ex.getMessage()));
```

This is far more readable and maintainable than nested `Future.get()` calls.

Q16. What is the difference between `thenApply`, `thenCompose`, and `thenAccept`?

Answer: `thenApply` transforms the result of a future synchronously using a function and returns a new `CompletableFuture` with the transformed value. `thenCompose` chains another asynchronous operation that itself returns a `CompletableFuture`, effectively flattening nested futures from `CompletableFuture<CompletableFuture<T>>` to `CompletableFuture<T>`. `thenAccept` consumes the result without returning a value, producing `CompletableFuture<Void>`, and is used for side effects such as logging or sending notifications. Each has an async variant, `thenApplyAsync`, `thenComposeAsync`, and `thenAcceptAsync`, which run the stage on a separate thread from the default `ForkJoinPool.commonPool()` or a supplied executor. Real-time example: fetching a user then mapping to a DTO is `thenApply`. Fetching a user then asynchronously fetching their orders is `thenCompose`. Sending a notification email after an order is placed is `thenAccept`.

Q17. How do `handle`, `exceptionally`, and `whenComplete` differ in `CompletableFuture`?

Answer: `exceptionally` is invoked only when an exception occurs in the previous stage. It receives the exception and returns a fallback value, allowing the pipeline to continue successfully. `handle` is invoked whether the previous stage succeeded or failed, receiving both the result and the exception. It can produce a new value or recover from errors, making it useful for unified success and error processing. `whenComplete` is a side-effect stage that observes the result and exception without changing them; it is useful for logging, metrics, or cleanup and returns the original result or rethrows the original exception. Use `exceptionally` for simple recovery, `handle` when you need to transform both outcomes, and `whenComplete` for non-transforming observation. Real-time example: a payment processing chain might use `exceptionally(ex -> PaymentStatus.RETRY)` to return a retry status. It might use `handle((result, ex) -> ex != null ? logAndReturnFallback(ex) : result)` to log failures and return a fallback. It might use `whenComplete((result, ex) -> paymentMetrics.record(result, ex))` to record metrics without changing the flow.

Q18. What is task scheduling in Spring Boot, and what annotations are used?

Answer: Spring Boot task scheduling allows methods to run periodically or at specific times using the `@Scheduled` annotation. The method is triggered by a `TaskScheduler`, which is auto-configured when scheduling is enabled. Common attributes include `fixedDelay`, which waits for the previous execution to finish plus the delay; `fixedRate`, which schedules executions at a fixed interval regardless of completion time; `cron`, which uses a cron expression for complex schedules; and `initialDelay`, which delays the first execution. The `@EnableScheduling` annotation is required on a configuration class to activate the scheduling infrastructure. By default, Spring Boot configures a single-threaded scheduler, which means one long-running task can delay others unless a custom thread pool is configured. Real-time example: a daily report generation service uses `@Scheduled(cron = "0 0 1 * * *")` on a method named `generateDailyReport()` so it runs at 1 AM every day. Another service uses `@Scheduled(fixedRate = 60000)` to poll an external API every minute for status updates.

Q19. What is the difference between `@EnableScheduling` and `@EnableAsync`?

Answer: `@EnableScheduling` enables the scheduling infrastructure for `@Scheduled` methods, which run on threads managed by a `TaskScheduler`. It is used for periodic or time-based execution. `@EnableAsync` enables the asynchronous execution infrastructure for `@Async` methods, which run on threads managed by a `TaskExecutor`. It is used for background execution when a method is called. They serve fundamentally different purposes: scheduling triggers execution based on time, while async executes methods in the background when invoked by code. Both can be enabled together in the same application. Each uses a different executor type by default, and custom executors can be configured for either. Real-time example: an application has a `@Scheduled` method that runs every hour to clean up stale sessions, enabled by `@EnableScheduling`. It also has an `@Async` method that sends welcome emails when a user registers, enabled by `@EnableAsync`. The cleanup runs on a scheduler thread, while the email sending runs on an async executor thread when triggered by user registration.

Q20. What are the rules for using `@Async` in Spring Boot?

Answer: `@Async` methods must be called from another bean because Spring uses proxies to intercept the call. Self-invocation from within the same class bypasses the proxy and runs synchronously. The method should return `void` for fire-and-forget tasks, `Future<T>` for legacy compatibility, or `CompletableFuture<T>` for modern async composition. Exceptions thrown in an `@Async` method are not propagated to the caller unless the caller inspects the returned `Future` or `CompletableFuture`. A custom executor should be configured for production because the default `SimpleAsyncTaskExecutor` creates a new thread for every task, which is not a true pool. The executor bean can be named and referenced via the `@Async("beanName")` qualifier. Real-time example: a notification service has `@Async public`

`CompletableFuture<Void> sendEmail(String to, String subject)`. A registration service injects the notification service and calls `notificationService.sendEmail(user.getEmail(), "Welcome")`. The proxy intercepts the call, runs it on the async executor, and the registration service continues immediately. If `sendEmail` were called from within the notification service itself, it would run synchronously.

Q21. How do you configure custom thread pools for @Async and @Scheduled in Spring Boot?

Answer: For `@Async`, create a `ThreadPoolTaskExecutor` bean. Name it `applicationTaskExecutor` to make it the default async executor, or give it a custom name and reference it with `@Async("beanName")`. For `@Scheduled`, create a `ThreadPoolTaskScheduler` bean or configure `spring.task.scheduling.pool.size` to set the scheduler pool size. A well-configured executor should specify `corePoolSize`, `maxPoolSize`, `queueCapacity`, a `ThreadFactory` that names threads for debugging, and graceful shutdown settings such as `setWaitForTasksToCompleteOnShutdown(true)` and `setAwaitTerminationSeconds(60)`. Naming threads is important because thread dumps show meaningful names like `async-email-1` instead of `pool-3-thread-1`, making production debugging much easier. Real-time example:

```
@Bean(name = "emailExecutor")
public Executor emailExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(5);
    executor.setMaxPoolSize(20);
    executor.setQueueCapacity(100);
    executor.setThreadNamePrefix("email-");
    executor.setWaitForTasksToCompleteOnShutdown(true);
    executor.setAwaitTerminationSeconds(30);
    executor.initialize();
    return executor;
}
```

Then use `@Async("emailExecutor")` on email sending methods. This prevents the default executor from spawning unlimited threads.

Q22. What is the Tomcat server threading model?

Answer: Tomcat uses a multi-threaded model to process incoming HTTP requests. An acceptor thread listens on the configured port for new TCP connections. A poller thread manages non-blocking I/O for accepted connections, especially in NIO and NIO2 connectors. When a request arrives, a worker thread from the thread pool is assigned to process it. The worker thread executes the servlet request, invokes the Spring Boot dispatcher servlet, runs application logic, generates a response, and returns to the pool. The maximum number of worker threads is controlled by `maxThreads`. Connections beyond the current worker capacity wait in an internal queue until a thread becomes available, and further connections are queued at the OS level up to

`acceptCount`. For I/O-bound applications, async servlets release the worker thread while waiting for external I/O, allowing Tomcat to handle more concurrent connections than `maxThreads`. Real-time example: a Spring Boot REST API with default Tomcat settings has `maxThreads=200`. Each request performs some business logic and database access. If a request takes 200 milliseconds, the theoretical throughput is about 1000 requests per second. If the application makes a slow external API call synchronously, all 200 threads block and the server becomes unresponsive even though the CPU is idle.

Q23. What are the key Tomcat connector tuning parameters?

Answer: Key parameters include `maxThreads`, the maximum number of request-processing worker threads; `minSpareThreads`, the minimum number of threads kept alive to handle sudden traffic; `maxConnections`, the maximum number of concurrent connections the poller can hold open; `acceptCount`, the OS-level socket backlog queue size when `maxConnections` is reached; `connectionTimeout`, how long Tomcat waits for the request line after accepting a connection; `keepAliveTimeout`, how long idle keep-alive connections remain open; and `connectionUploadTimeout`, timeout for reading request bodies. In Spring Boot, these map to properties under `server.tomcat.*`. Tuning requires balancing memory, CPU, and expected concurrent load. Increasing `maxThreads` alone does not fix slow application logic; it only increases the number of concurrent slow operations. Real-time example: a microservice runs on a 2-CPU container with 2 GB memory. Setting `maxThreads=500` wastes memory on thread stacks and increases context switching. After profiling, the optimal setting is `maxThreads=50`, `minSpareThreads=10`, `acceptCount=100`, and `connectionTimeout=20000`. The service handles load efficiently without memory pressure.

Q24. What is the relationship between Tomcat worker threads and Spring Boot application threads?

Answer: When a request arrives at Tomcat, a Tomcat worker thread invokes the Spring Boot dispatcher servlet and executes the application code on the same thread unless async processing is explicitly used. This means application logic typically runs within the Tomcat thread pool. If the application makes a blocking call, such as a database query or external HTTP request, the Tomcat worker thread remains occupied until the call completes. For CPU-bound workloads this is acceptable, but for I/O-bound workloads it limits scalability. Using `@Async`, `CompletableFuture`, `DeferredResult`, `Callable`, or reactive programming moves work off the Tomcat worker thread, allowing the same thread pool to handle many more concurrent connections. Real-time example: a payment endpoint calls a bank API that takes 2 seconds. With synchronous processing, a Tomcat pool of 50 threads can handle only about 25 requests per second. By returning `CompletableFuture<ResponseEntity<PaymentResult>>` and running the bank call on a dedicated async executor, the same 50 Tomcat threads can accept requests and dispatch them asynchronously, increasing throughput to hundreds of requests per second.

Scenario based questions

S1. Two threads increment a shared counter, but the final value is less than expected. What is wrong?

Answer: This is a classic race condition caused by a non-atomic read-modify-write sequence. The increment operation `count++` compiles to multiple bytecode instructions: read the current value, add one, and write back. Without synchronization, both threads can read the same value concurrently, increment it independently, and write back the same result, causing one update to be lost. The fix is to use `AtomicInteger` for lock-free atomic increments, or synchronize the increment method or block. `AtomicInteger` uses compare-and-swap hardware instructions and is more efficient than locking for simple counters. In a distributed system with multiple JVMs, a distributed counter or database sequence may be needed. Real-time example: a REST endpoint tracks total downloads using a shared `int downloadCount`. Under a load test of 10,000 concurrent downloads, the reported count is only 8,234 because updates were lost. Replacing `int downloadCount` with `AtomicInteger downloadCount = new AtomicInteger(0);` and using `downloadCount.incrementAndGet()` produces the correct 10,000 count. For higher throughput across multiple counters, `LongAdder` may be even better because it reduces contention across internal cells.

S2. A `synchronized` method causes poor performance under load. How do you optimize it?

Answer: I first profile to confirm that synchronization is the actual bottleneck and identify what portion of the method truly needs protection. The first optimization is to reduce the synchronized scope to the smallest critical section that accesses shared mutable state. If the method is mostly read-only with occasional writes, I replace it with a `ReadWriteLock` where many threads can hold the read lock simultaneously and only writers block readers. I also evaluate concurrent data structures such as `ConcurrentHashMap`, `ConcurrentLinkedQueue`, `CopyOnWriteArrayList`, or `LongAdder`, which provide fine-grained locking or lock-free algorithms. If contention remains high, I consider data structure sharding, where the shared state is split into multiple independently locked segments, or actor-style single-threaded ownership where one thread owns the state and others communicate via queues. Real-time example: a reporting service has a synchronized method that reads and updates a large in-memory cache map. Reads are 99 percent of operations. Replacing the method-level `synchronized` with a `ReadWriteLock` increases read throughput by a factor of ten because multiple threads can read concurrently. For even better performance, switching to `ConcurrentHashMap` eliminates manual locking entirely.

S3. An application creates a new thread for every request and eventually crashes. What is the fix?

Answer: The application is using `new Thread()` per request, which has high creation overhead and consumes operating system resources. Threads have stack memory, kernel bookkeeping structures, and context-switching costs. Creating thousands of threads exhausts native memory or

hits OS limits, causing `OutOfMemoryError: unable to create new native thread` or crashes. The fix is to use a thread pool through `ExecutorService` or `ThreadPoolExecutor`, which reuses a bounded number of threads. I configure `corePoolSize`, `maxPoolSize`, `queueCapacity`, a rejection policy, and a named `ThreadFactory`. For short-lived tasks, a fixed or cached bounded pool is appropriate. For production, I also implement graceful shutdown. Real-time example: an integration service receives webhooks and spawns `new Thread(() -> processWebhook(payload)).start()` for each. During a burst of 50,000 webhooks, the JVM creates too many threads and the Linux process limit is reached, crashing the service. Replacing this with:

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(20, 100, 60L,
TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(1000),
    new ThreadFactoryBuilder().setNameFormat("webhook-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy());
```

stabilizes the service. The bounded queue and `CallerRunsPolicy` provide backpressure by making the caller process tasks when saturated.

S4. A `ThreadPoolExecutor` rejects tasks under heavy load. How do you handle it?

Answer: Task rejection means the pool has reached `maximumPoolSize` and the work queue is full. I first determine whether the rejection is a normal capacity signal or a problem. If the system is intentionally designed with backpressure, `CallerRunsPolicy` is often the best choice because it slows down the producer naturally and prevents overload. If rejections indicate the pool is undersized, I increase `maxPoolSize` and queue capacity within memory and CPU limits, or scale horizontally by adding application instances. I also review task duration because long-running tasks reduce effective throughput. I log and monitor rejected tasks with a custom `RejectedExecutionHandler` to track patterns. A custom handler can also persist failed tasks to a dead-letter queue for retry. Real-time example: during a flash sale, an order processing executor with `corePoolSize=50`, `maxPoolSize=50`, and a queue of 100 starts rejecting orders. I increase `maxPoolSize` to 150 and queue capacity to 500. I add a custom rejection handler that publishes rejected orders to a Kafka topic so a separate consumer can retry them once the load subsides. This prevents lost orders while protecting the main service.

S5. A `Callable` task submitted to an `ExecutorService` throws an exception, but the caller never sees it. Why?

Answer: Exceptions thrown in `Callable.call()` are captured inside the returned `Future` object, not propagated immediately. If the caller never calls `Future.get()`, the exception remains trapped in the future and is effectively swallowed. With `ExecutorService.execute(Runnable)`, uncaught exceptions may be lost unless an `UncaughtExceptionHandler` is configured. The fix is to always handle the `Future` returned by `submit()`. With plain `Future`, call `get()` and catch `ExecutionException` to inspect

the underlying cause. With `CompletableFuture`, exceptions propagate through the chain and can be handled with `exceptionally`, `handle`, or `whenComplete`. Real-time example: a batch job submits 100 `Callable` tasks to process payments and only stores the futures in a list. Because the code never iterates the list and calls `get()`, failed payments are silently ignored and customers are not charged. The fix collects results and logs failures:

```
for (Future<PaymentResult> future : futures) {
    try {
        PaymentResult result = future.get();
        if (!result.isSuccess()) log.warn("Payment failed: {}", result);
    } catch (ExecutionException e) {
        log.error("Payment task failed", e.getCause());
    }
}
```

Alternatively, using `CompletableFuture` with `exceptionally` provides cleaner centralized error handling.

S6. You have two independent API calls to make, and then you need to combine their results. How do you implement it with `CompletableFuture`?

Answer: I use `CompletableFuture.supplyAsync()` for each independent call, passing a custom executor for better control over the thread pool. I then combine the results using `thenCombine` if I need to merge exactly two futures with a combining function, or `CompletableFuture.allOf` followed by joining each future for more than two. `thenCombine` runs a `BiFunction` once both futures complete. `allOf` returns a future that completes when all provided futures complete, but it returns `Void`, so I keep references to the original futures to retrieve their results. I handle exceptions per future with `exceptionally` or `handle` so one failure does not unnecessarily break the entire response. Real-time example: an order summary screen needs user profile and order history from two separate microservices. Synchronous calls take 300 ms plus 500 ms, totaling 800 ms. Using:

```
CompletableFuture<UserProfile> profileFuture = CompletableFuture.supplyAsync(
    () -> userService.getProfile(userId), ioExecutor);
CompletableFuture<List<Order>> ordersFuture = CompletableFuture.supplyAsync(
    () -> orderService.getHistory(userId), ioExecutor);

CompletableFuture<Summary> summary = profileFuture.thenCombine(ordersFuture,
    (profile, orders) -> new Summary(profile, orders));
```

reduces the total time to approximately 500 ms, the duration of the slower call. If one service fails, an `exceptionally` fallback can return cached or partial data.

S7. A `CompletableFuture` chain uses `thenApply` but the transformation itself makes a blocking database call. What is the risk?

Answer: The risk is that the blocking database call runs on the thread that completed the previous stage. If the previous stage ran on the `ForkJoinPool.commonPool`, the common pool thread

blocks, reducing the pool's effective capacity and potentially starving other async operations that use it. Even if the previous stage ran on the caller's thread, blocking there can delay response processing. The fix is to use `thenApplyAsync` or `thenComposeAsync` and supply an executor designed for blocking I/O with enough threads. Better yet, use `thenComposeAsync` to call an async database client that returns a `CompletableFuture`, such as a reactive database driver. For Spring Data, returning reactive types or using `@Async` services can also help. Real-time example: a chain fetches user details and then queries a slow CRM database inside `thenApply`:

```
fetchUser(userId)
    .thenApply(user -> crmRepository.findByEmail(user.getEmail()))
    .thenApply(crm -> enrich(user, crm));
```

During load, the common pool threads block on the CRM query, causing other futures to queue. The fix moves the CRM call to `thenComposeAsync` with a dedicated executor:

```
fetchUser(userId)
    .thenComposeAsync(user -> crmAsyncClient.findByEmail(user.getEmail()),
blockingIoExecutor)
    .thenApply(crm -> enrich(user, crm));
```

This isolates blocking work from the common pool and restores responsiveness.

S8. A scheduled task in Spring Boot is not running on time after a long-running previous execution. Why?

Answer: If the task is configured with `fixedDelay`, the next execution intentionally waits until the previous execution finishes plus the delay. This behavior prevents overlap and is correct for sequential tasks. If the requirement is to start at fixed wall-clock intervals regardless of duration, `fixedRate` should be used instead. With `fixedRate`, the scheduler triggers the next execution at the interval, but if the task takes longer than the interval, executions can queue up unless configured otherwise. For cron expressions, the next scheduled time is based on clock time, not previous execution duration. Another common issue is that Spring Boot's default scheduler uses a single thread, so a long-running scheduled task can delay other scheduled tasks. The fix is to configure a `ThreadPoolTaskScheduler` with a larger pool or assign a dedicated scheduler to long tasks. Real-time example: a nightly report generation job sometimes runs for 30 minutes, and an hourly cleanup task starts late. Both share the default single-threaded scheduler. I create a `ThreadPoolTaskScheduler` named `reportScheduler` with pool size 5 and annotate the report job with `@Scheduled(fixedDelay = 60 * 60 * 1000, scheduler = "reportScheduler")`. The cleanup task keeps the default scheduler. Now both tasks run independently and on time.

S9. A Spring Boot @Async method runs synchronously when called from within the same class. Why?

Answer: Spring uses proxies, typically JDK dynamic proxies or CGLIB subclasses, to implement `@Async`. When a method is called from within the same class, the call bypasses the proxy and invokes the target method directly. Because the proxy is not involved, Spring cannot intercept the

call and schedule it on the async executor. The fix is to move the `@Async` method to another Spring bean and inject that bean, so calls go through the proxy. This is the same self-invocation limitation that affects `@Transactional` and `@Scheduled` annotations. Real-time example: an `OrderService` has a public `@Async CompletableFuture<Void> sendNotification(Order order)` method and another method `processOrder(Order order)` that calls `sendNotification(order)`. Notifications are sent synchronously, blocking order processing. The fix extracts `NotificationService` as a separate bean, injects it into `OrderService`, and calls `notificationService.sendNotification(order)`. The proxy intercepts the call and runs it on the async executor, so `processOrder` returns immediately.

S10. A Spring Boot scheduled job works in development but runs twice in production. What causes this?

Answer: In development there is usually one application instance, so the scheduled job runs once. In production there are often multiple instances deployed, and each JVM runs the scheduled job independently, causing duplicate execution. Spring Boot scheduling is per-JVM by default. To fix this, use a distributed scheduler such as ShedLock, Quartz with JDBC job store, or a Kubernetes CronJob that runs as a single Pod. Another possibility is that multiple application contexts are started in the same JVM, which can happen with incorrect Spring Boot test configurations or nested contexts. I check the deployment topology and ensure exactly one instance executes the job. ShedLock is a lightweight solution that uses a database or Redis lock. Real-time example: a daily invoice generation job runs on two Kubernetes pods and creates duplicate invoices. I add the ShedLock dependency, create a lock table, and annotate the method:

```
@Scheduled(cron = "0 0 2 * * *")
@SchedulerLock(name = "generateInvoices", lockAtMostFor = "PT2H")
public void generateInvoices() { ... }
```

Only one pod acquires the lock and runs the job. Duplicates stop, and if the lock holder crashes, the lock expires after two hours so another pod can take over.

S11. A Tomcat server becomes unresponsive under heavy load. What do you check?

Answer: I check whether all Tomcat worker threads are busy handling requests. I monitor Tomcat metrics via JMX or Spring Boot actuator for busy threads, max threads, and queued connections. If all worker threads are occupied by slow requests, new connections queue up until `acceptCount` is reached and then time out. The root cause is usually slow application logic such as external API calls, database queries, or heavy computation that blocks the worker thread. The fix is to identify the slow operations and make them asynchronous, moving work off the Tomcat thread using `CompletableFuture`, `DeferredResult`, `Callable`, or reactive programming. Increasing `maxThreads` may help briefly but does not fix slow logic and increases memory usage. Real-time example: during a sale, the checkout API calls a payment gateway that takes 3 seconds synchronously. With default `maxThreads=200`, all threads block and users see 504 gateway

timeouts. Profiling the thread dump shows every worker waiting on the payment gateway socket. I refactor the endpoint to return `DeferredResult<ResponseEntity<PaymentResult>>`, run the gateway call on a dedicated async executor, and complete the result when the gateway responds. Tomcat workers are released immediately, and the server handles 10 times more concurrent connections.

S12. A Spring Boot application with default Tomcat config runs out of memory under load. How do you tune it?

Answer: I first determine whether the memory issue is heap or native thread memory. Default Tomcat `maxThreads` is 200, and each thread consumes stack memory, typically 1 MB by default. If many threads are idle or the workload is small, reducing `maxThreads` frees significant native memory. I set `minSpareThreads` to a reasonable value that avoids thread creation overhead without keeping too many threads alive. I tune `acceptCount` and `maxConnections` to prevent excessive queued connections from consuming memory. I ensure `connectionTimeout` and `keepAliveTimeout` are not holding idle keep-alive connections too long. For I/O-bound workloads, I reduce thread count and use async servlets so fewer worker threads are needed. Real-time example: a microservice with default `maxThreads=200` runs on a small container with 1 GB memory. Each Tomcat thread uses about 1 MB stack, consuming 200 MB just for threads, plus heap, plus other native memory. I reduce `maxThreads` to 50 and `minSpareThreads` to 10 in `application.yml`:

```
server.tomcat.threads.max: 50
server.tomcat.threads.min-spare: 10
```

I also convert blocking downstream calls to async processing. Memory usage drops by over 50 percent and the container runs stably.

S13. A `@Scheduled` method throws an exception, and subsequent executions stop. Why?

Answer: An unhandled exception in a scheduled task does not stop the Spring scheduler process itself, but it can stop the specific scheduled task if the scheduler is single-threaded and the exception propagates out of the method. By default, Spring Boot configures a single scheduler thread that handles all `@Scheduled` methods, so an exception in one task or a long-running task can delay others. The fix is to catch exceptions inside the scheduled method body and log them, preventing the exception from bubbling out. Alternatively, configure a custom `ErrorHandler` on the scheduler. For critical tasks, I wrap the entire method body in try-catch and may send alerts on failures. Real-time example: an hourly cache refresh task throws a `NullPointerException` because an external service briefly returns null. Because the exception is uncaught, the scheduler thread may behave unpredictably and the cache stops refreshing. I add:

```
@Scheduled(fixedRate = 60 * 60 * 1000)
public void refreshCache() {
    try {
        cacheService.refresh();
```

```
    } catch (Exception e) {
        log.error("Cache refresh failed", e);
        alertService.send("Cache refresh failed: " + e.getMessage());
    }
}
```

Now transient failures are logged and alerted, but the task continues to run every hour.

S14. You need to run a task after another task completes, but both are asynchronous. Which `CompletableFuture` method do you use?

Answer: I use `thenCompose` if the second task depends on the result of the first and itself returns a `CompletableFuture`. This flattens `CompletableFuture<CompletableFuture<T>>` into `CompletableFuture<T>`. I use `thenApply` if the second step is a synchronous transformation of the first result. If the two tasks are independent and I need both results together, I use `thenCombine`. For dependent sequential async calls, `thenCompose` is the correct choice because it avoids nested futures and creates a clean linear pipeline. Real-time example: a checkout flow first validates inventory asynchronously and then charges payment asynchronously. The payment step needs the validation result:

```
CompletableFuture<OrderResult> result = validateInventory(order)
    .thenCompose(valid -> {
        if (!valid) return
CompletableFuture.completedFuture(OrderResult.outOfStock());
        return chargePayment(order);
    })
    .thenCompose(payment -> createShipment(order, payment))
    .exceptionally(ex -> OrderResult.failure(ex.getMessage()));
```

The inventory validation, payment, and shipment run sequentially, but each step is non-blocking. The entire chain returns a single future that completes when the shipment is created.

S15. A `CompletableFuture.allOf` pipeline completes but you cannot easily get individual results. How do you solve it?

Answer: `CompletableFuture.allOf` returns `CompletableFuture<Void>` because it only signals that all futures are done. To collect results, I keep references to the original futures and call `join()` or `get()` on each after `allOf` completes. A cleaner approach for a small fixed number of futures is `thenCombine`. For a dynamic list, I store futures in a `List<CompletableFuture<T>>`, call `CompletableFuture.allOf(futures.toArray(new CompletableFuture[0]))`, and then map the list with `future.join()`. I wrap the operation in try-catch because `join()` throws `CompletionException` for unchecked exceptions. Real-time example: I need product, pricing, and inventory from three independent services:

```
CompletableFuture<Product> productFuture = supplyAsync(() ->
productService.get(id), ioExecutor);
CompletableFuture<Price> priceFuture = supplyAsync(() -> pricingService.get(id),
ioExecutor);
CompletableFuture<Stock> stockFuture = supplyAsync(() ->
```

```
inventoryService.get(id), ioExecutor);

CompletableFuture<Void> all = CompletableFuture.allOf(productFuture,
priceFuture, stockFuture);
CompletableFuture<ProductDetail> detail = all.thenApply(v -> new ProductDetail(
    productFuture.join(), priceFuture.join(), stockFuture.join()))
    .exceptionally(ex -> ProductDetail.error(ex.getCause()));
```

This waits for all three and combines their results. If any service fails, `join()` throws and `exceptionally` handles it.

S16. A Spring Boot application needs to run a background job every 10 seconds, but the job must not overlap with itself. How do you ensure this?

Answer: I use `fixedDelay` instead of `fixedRate` so the next execution only starts after the previous one finishes plus the delay. This guarantees no overlap because the scheduler waits for completion. If the requirement is to run at fixed wall-clock intervals and the task duration is always shorter than the interval, `fixedRate` with a single scheduler thread also prevents overlap because the scheduler cannot run the next execution until the thread is free. For clustered deployments, I add a distributed lock such as ShedLock to ensure only one instance runs the job. For cases where `fixedRate` or cron is required but overlap must be prevented, I implement a task-level lock inside the method using `ReentrantLock` or a boolean flag. Real-time example: a data export job takes between 5 and 20 seconds. Using `@Scheduled(fixedRate = 10000)` causes overlapping exports and database contention. I switch to `@Scheduled(fixedDelay = 10000)` so the next export starts 10 seconds after the previous one completes. For a cluster of three instances, I also add ShedLock so only one instance exports at a time.

S17. A thread appears to hang forever waiting for a lock. How do you diagnose it?

Answer: I generate a thread dump using `jstack <pid>`, `kill -3 <pid>`, or the JVM diagnostic command `jcmd <pid> Thread.print`. I look for threads in `BLOCKED` state and identify which monitor they are waiting to acquire. I find the thread that currently holds the lock and examine what it is doing. If that thread is waiting for another lock held by yet another thread, I look for a deadlock cycle. I also check threads in `WAITING` or `TIMED_WAITING` to see if they are waiting for `notify()` that never arrives. In production, I use monitoring tools such as Datadog, Dynatrace, New Relic, or Java Flight Recorder to capture thread state and trends. Once the cause is identified, I fix synchronization logic, reduce lock scope, establish lock ordering to prevent deadlock, or replace locks with concurrent data structures. Real-time example: a payment service hangs during peak load. A thread dump shows `http-worker-15` blocked on a lock held by `http-worker-3`, which is waiting for a database connection currently held by `http-worker-15`. This is a circular wait deadlock caused by acquiring the lock before the database connection. I refactor to acquire the database connection first, then the business lock, breaking the cycle.

S18. A method is annotated with `@Async` but returns a normal object instead of `CompletableFuture`. What happens?

Answer: The method still runs asynchronously because the Spring proxy intercepts the call and schedules it on the executor, but the caller receives the return value immediately, not the actual computed result. Since the async task has not finished when the proxy returns, the caller usually receives `null` for object returns or a partially initialized object. The caller cannot wait for completion or handle exceptions through the returned value. Best practice is to return `CompletableFuture<T>` for async methods that produce a result, or `void` for fire-and-forget tasks. Returning a normal object defeats the purpose of async composition and makes error handling nearly impossible. Real-time example: a service has `@Async public User fetchUser(String id)`. A caller invokes it and immediately uses the returned `User`, but the async method is still querying the database. The caller receives `null` and throws `NullPointerException`. The fix changes the method to `@Async public CompletableFuture<User> fetchUser(String id)` and the caller uses `fetchUser(id).thenApply(user -> ...)` or `fetchUser(id).join()` when the result is needed.

S19. A `CompletableFuture` chain should fail fast if any stage takes too long. How do you add a timeout?

Answer: Java 9 introduced `orTimeout` and `completeOnTimeout` on `CompletableFuture`. I use `future.orTimeout(duration, TimeUnit)` to complete the future exceptionally if it does not finish in time. I use `completeOnTimeout(defaultValue, duration, TimeUnit)` to provide a fallback value instead of failing. In Java 8, I can schedule a `CompletableFuture.completeExceptionally` call with a `ScheduledExecutorService` after the timeout. When combining multiple futures, I apply timeout to individual stages so the failure is isolated and other independent stages can still succeed. Real-time example: a recommendation service calls three external providers in parallel, and the overall response must return within 2 seconds. I apply `orTimeout(800, TimeUnit.MILLISECONDS)` to each provider future and use `exceptionally` to return a default recommendation if one times out:

```
CompletableFuture<Recommendations> recs = providerA.get(userId).orTimeout(800,
MILLISECONDS)
    .exceptionally(ex -> defaultRecommendations);
```

The final response is assembled from whichever providers respond in time, improving availability while meeting the latency requirement.

S20. How do you gracefully shut down a `ThreadPoolExecutor` in production?

Answer: I call `shutdown()` to stop accepting new tasks while allowing already submitted tasks to finish. I then call `awaitTermination(timeout, TimeUnit)` to wait for running tasks to complete. If tasks are still running after the timeout, I call `shutdownNow()` to attempt to interrupt them, collect the list of tasks that were waiting to run, and decide whether to persist them for retry or log them. I never call `shutdownNow()` first because it cancels tasks abruptly and may leave data in an inconsistent state. I also ensure task implementations respond to interruption by checking `Thread.currentThread().isInterrupted()` and cleaning up resources. In Spring Boot, I configure the `ThreadPoolTaskExecutor` with `setWaitForTasksToCompleteOnShutdown(true)` and `setAwaitTerminationSeconds(60)`. Real-time example: a report generation service is being redeployed. Without graceful shutdown, a report in progress is killed mid-write, leaving partial data in the database. I configure:

```
executor.setWaitForTasksToCompleteOnShutdown(true);
executor.setAwaitTerminationSeconds(60);
```

On shutdown, Spring waits up to 60 seconds for active reports to finish before terminating the JVM. Reports that did not start are logged and retried by the next deployment.

S21. A Spring Boot controller calls a slow external API synchronously. How do you prevent Tomcat thread exhaustion?

Answer: I make the controller return an asynchronous result type such as `CompletableFuture<ResponseEntity<T>>`, `DeferredResult<T>`, or `Callable<T>`. This releases the Tomcat worker thread immediately and allows the response to be completed later when the external API responds. The external call should run on a dedicated async executor with enough threads for blocking I/O, not on the `ForkJoinPool.commonPool`. I configure the async timeout and error handling so the client receives a proper response or timeout error instead of a generic server error. For fully reactive stacks, returning `Mono` or `Flux` with WebClient also releases the Tomcat thread. Real-time example: a credit check endpoint calls a bureau API that takes 3 seconds. With 200 Tomcat threads, the application handles about 60 requests per second. I change the controller to:

```
@GetMapping("/credit-check/{id}")
public CompletableFuture<ResponseEntity<Decision>> checkCredit(@PathVariable
String id) {
    return CompletableFuture.supplyAsync(() -> bureauService.check(id),
bureauExecutor)
        .thenApply(decision -> ResponseEntity.ok(decision))
        .exceptionally(ex ->
ResponseEntity.status(503).body(Decision.unavailable()));
}
```

Tomcat threads are released after scheduling the work, and the same pool can accept many more requests. Throughput increases to 500 requests per second.

S22. What happens if `corePoolSize` equals `maximumPoolSize` in a `ThreadPoolExecutor`?

Answer: The pool has a fixed size. It creates threads up to `corePoolSize` for incoming tasks and then queues additional tasks. Extra threads beyond the core size are never created because the maximum equals the core. If the queue is bounded and fills up, tasks are rejected according to the rejection policy. If the queue is unbounded, tasks can accumulate indefinitely, risking out-of-memory errors if the producer is faster than the consumer. This configuration is useful when you want a predictable number of threads and rely on the queue for buffering. It is commonly used with `Executors.newFixedThreadPool(n)`, which creates a fixed pool with an unbounded `LinkedBlockingQueue`. For production, I prefer a bounded queue with a meaningful rejection policy rather than an unbounded queue that can silently exhaust memory. Real-time example: an order validation service uses `Executors.newFixedThreadPool(20)`, which has 20 threads and an unbounded queue. During a traffic spike, the producer submits millions of validation tasks faster than they are processed. The queue grows until the JVM runs out of memory. I replace it with an explicit `ThreadPoolExecutor` using a bounded `ArrayBlockingQueue` of 1000 and `CallerRunsPolicy`. This provides backpressure, keeps memory stable, and prevents cascading failure.

S23. A scheduled task needs to run at 2 AM in a specific timezone. How do you configure it?

Answer: I use the `cron` attribute of `@Scheduled` with the `zone` attribute to specify the timezone. For example, `@Scheduled(cron = "0 0 2 * * *", zone = "America/New_York")` runs at 2 AM Eastern Time every day. Without the `zone` attribute, the cron expression uses the server's default timezone, which can cause incorrect execution times if the server timezone changes or if instances run in different regions. I always set the `zone` explicitly for time-sensitive scheduled tasks and document the timezone in the code and runbooks. I also consider daylight saving time changes when choosing the timezone. Real-time example: a financial reconciliation report must run at 2 AM New York time to match the market close. The application servers run in UTC. Without the zone, the report runs at 2 AM UTC, which is 10 PM the previous day in New York during standard time. I annotate the method with:

```
@Scheduled(cron = "0 0 2 * * *", zone = "America/New_York")
public void runReconciliation() { ... }
```

The task now runs at the correct local time in New York, adjusting for daylight saving time automatically.

S24. A `@Scheduled` task processes a large batch and should not delay other scheduled tasks. How do you design it?

Answer: I configure a dedicated `TaskScheduler` with a larger thread pool for the long-running task, or I move the actual batch processing to an `@Async` method so the scheduler thread returns immediately after triggering the work. The scheduler thread should be lightweight and only

responsible for triggering work; heavy processing should run in a separate executor. If the batch is too large for the application, I consider running it as a Kubernetes CronJob or an external scheduler that can scale independently. I also monitor task duration to detect regressions and alert if the batch exceeds expected time. Real-time example: a nightly aggregation task takes 2 hours and delays all other scheduled tasks because the default scheduler has one thread. I create a `ThreadPoolTaskScheduler` named `batchScheduler` with pool size 5:

```
@Bean("batchScheduler")
public TaskScheduler batchScheduler() {
    ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
    scheduler.setPoolSize(5);
    scheduler.setThreadNamePrefix("batch-sched-");
    return scheduler;
}
```

I annotate the aggregation method with `@Scheduled(cron = "0 0 1 * * *", scheduler = "batchScheduler")` and have the method delegate to an `@Async` executor for the actual processing. Other scheduled tasks are no longer blocked by the long batch.

S25. You have an `ExecutorService` and want to know when all tasks are done. How do you do it?

Answer: I submit all tasks and collect the returned `Future` objects. I iterate over the collection and call `get()` on each, which waits for completion and surfaces any `ExecutionException`. Alternatively, I use `ExecutorService.invokeAll(Collection<? extends Callable<T>>)` to submit a collection of `Callable` tasks and wait for all to complete; it returns a list of futures in the same order as the input. With `CompletableFuture`, I use `CompletableFuture.allOf` or collect futures in a list and call `join()` after waiting. For manual coordination, I can use a `CountDownLatch` initialized with the task count; each task calls `countDown()` when done, and the main thread calls `await()`. Real-time example: a report generation job fans out 50 subtasks to an executor to fetch data from different regions. I use:

```
List<Future<RegionData>> futures = executor.invokeAll(tasks);
for (Future<RegionData> future : futures) {
    try {
        report.addRegionData(future.get());
    } catch (ExecutionException e) {
        log.error("Region task failed", e.getCause());
        report.markRegionFailed(e.getCause().getMessage());
    }
}
```

`invokeAll` blocks until all tasks complete, and the loop handles both successful results and failures. This ensures the report includes all available data and logs any regional failures.